

# SMARTLOGIX - Analisis de Patrones y Arquitectura

## Justificacion de microservicios

SMARTLOGIX separa inventario, pedidos y envios en servicios independientes con bases de datos propias. Esto permite escalar, desplegar y evolucionar cada dominio sin afectar al resto. La comunicacion es sincrona via REST entre servicios y el BFF expone una API unificada al frontend.

## Arquitectura BFF

El bff-gateway (puerto 8080) centraliza llamadas y oculta URLs internas. LogisticaFacade agrega pedido, productos y envio para el dashboard sin duplicar reglas de negocio complejas. El frontend solo consume /api del BFF.

## Repository Pattern

Problema: acoplar servicios a JPA/SQL. Solucion: interfaces ProductoRepository, PedidoRepository, EnvioRepository. Usado en los tres microservicios para abstraer persistencia.

## Builder Pattern

Problema: construccion de Producto con muchos campos y defaults. Solucion: ProductoBuilder en ms-inventario con metodo desdeRequest().

## Factory Method

Problema: creacion de Pedido segun contexto (nuevo, rechazado). Solucion: PedidoFactory centraliza la instanciacion y estados iniciales.

## Facade Pattern

Problema: multiples llamadas REST desde pedidos a inventario y desde BFF a todos los MS. Solucion: InventarioFacade (ms-pedidos) y LogisticaFacade (bff-gateway) unifican operaciones.